

Phạm Xuân Quý – B22DCAT240
Lớp: E22CNPM03

NỘI DUNG BÁO CÁO

Phần 1 – Tổng quan về AI trong E-Commerce

Phần 2 – Cá nhân hóa & Hệ thống Gợi ý Sản phẩm

Phần 3 – Phân tích Hành vi & Chatbot RAG

Phần 4 – Tìm kiếm Thông minh & Quản lý Kho hàng

Phần 5 – Tổng hợp & Xu hướng Tương lai

PHẦN 1

Tổng Quan về AI trong Thương Mại Điện Tử

1.1. Giới thiệu

Thương mại điện tử (E-Commerce) đang bước vào một kỷ nguyên chuyển đổi sâu sắc, nơi Trí tuệ Nhân tạo (Artificial Intelligence — AI) không còn là lựa chọn tùy chọn mà trở thành yếu tố sống còn quyết định lợi thế cạnh tranh. Theo báo cáo của McKinsey & Company (2024), các doanh nghiệp e-commerce ứng dụng AI toàn diện ghi nhận mức tăng doanh thu trung bình 15–20% và giảm chi phí vận hành lên đến 30%. Những con số này phản ánh sức mạnh thực tiễn của AI khi được tích hợp đúng cách vào hệ sinh thái thương mại số.

AI trong e-commerce không chỉ đơn thuần là một công cụ tự động hóa — đây là bộ não thứ hai của nền tảng, có khả năng học hỏi từ hàng triệu điểm dữ liệu hành vi người dùng, tối ưu hoá luồng vận hành và cá nhân hóa trải nghiệm cho từng cá nhân mua sắm. Từ việc gợi ý sản phẩm, phát hiện gian lận thanh toán, tối ưu tồn kho đến hỗ trợ khách hàng 24/7 qua chatbot — AI hiện diện ở mọi điểm chạm (touchpoint) trong hành trình mua sắm.

1.2. Bức tranh thị trường AI E-Commerce toàn cầu

Quy mô thị trường AI trong lĩnh vực bán lẻ và thương mại điện tử toàn cầu được dự báo đạt 45,7 tỷ USD vào năm 2032, tăng từ mức 8,4 tỷ USD năm 2023, tương đương tốc độ tăng trưởng kép (CAGR) đạt 18,9%/năm (Grand View Research, 2024). Khu vực Châu Á — Thái Bình Dương, đặc biệt là Đông Nam Á với các thị trường như Việt Nam, Indonesia và Thái Lan, đang là vùng tăng trưởng nhanh nhất nhờ tỷ lệ thâm nhập smartphone cao và làn sóng số hóa mạnh mẽ.

Chỉ số	2022	2024	Dự báo 2028
Quy mô thị trường AI E-Commerce	6,2 tỷ USD	11,8 tỷ USD	31,4 tỷ USD
Tỷ lệ cá nhân hóa bằng AI	41%	67%	>85%

Chatbot AI xử lý CSKH	28%	54%	>70%
Tỷ lệ dùng AI trong logistics	33%	58%	>75%

Bảng 1.1 — Xu hướng tăng trưởng AI trong E-Commerce toàn cầu (Nguồn: Grand View Research, McKinsey 2024)

1.3. Các nhóm ứng dụng AI chủ chốt

Dựa trên phân tích các nền tảng e-commerce hàng đầu thế giới (Amazon, Alibaba, Shopee, Lazada), có thể phân loại ứng dụng AI thành 6 nhóm chiến lược chính:

(1) Cá nhân hóa & Gợi ý sản phẩm

Collaborative Filtering, Matrix Factorization, Deep Neural Networks phân tích lịch sử mua sắm, hành vi duyệt web để đề xuất sản phẩm phù hợp. Amazon ghi nhận 35 doanh thu đến từ hệ thống recommendation này.

(2) Phân tích hành vi & Phân khúc khách hàng

LSTM, Transformer phân tích chuỗi hành vi theo thời gian, phân nhóm khách hàng thành các segment có đặc điểm mua sắm khác nhau để cá nhân hóa chiến lược marketing.

(3) Chatbot & Trợ lý ảo (Conversational AI)

LLM kết hợp RAG (Retrieval-Augmented Generation) cung cấp tư vấn sản phẩm 24/7, giải đáp chính sách, hỗ trợ đặt hàng với khả năng hiểu ngữ cảnh và cá nhân hóa.

(4) Tìm kiếm thông minh (AI Search)

NLP + Semantic Search hiểu ý định tìm kiếm (search intent) vượt ra ngoài khớp từ khóa đơn thuần. Computer Vision cho phép tìm kiếm bằng hình ảnh (visual search).

(5) Dự báo nhu cầu & Tối ưu tồn kho

Time-series forecasting (ARIMA, Prophet, LSTM) dự báo xu hướng mua sắm, tự động tái đặt hàng, phân bổ kho thông minh giảm thiểu hàng tồn và tình trạng hết hàng.

(6) Phát hiện gian lận & Bảo mật

Anomaly Detection, Graph Neural Networks phân tích pattern giao dịch bất thường, xác thực danh tính, bảo vệ tài khoản người dùng trong thời gian thực.

PHẦN 2

Cá Nhân Hóa & Hệ Thống Gợi Ý Sản Phẩm

2.1. Kiến trúc Recommender System trong E-Commerce

Hệ thống gợi ý sản phẩm (Recommender System) là ứng dụng AI có tác động kinh doanh trực tiếp nhất trong e-commerce. Netflix tiết lộ rằng 80% nội dung được xem đến từ hệ thống recommendation; con số tương tự ở Amazon là 35% doanh thu. Có ba kiến trúc chính được triển khai trong thực tế:

Phương pháp	Đặc điểm & Ứng dụng
Collaborative Filtering	Học từ hành vi tập thể: 'Người dùng giống bạn cũng mua X'. Hiệu quả cao khi có lượng dữ liệu lớn, dễ bị cold-start problem với user/item mới.
Content-Based Filtering	Phân tích đặc điểm sản phẩm (thể loại, tác giả, tag) để gợi ý item tương tự. Phù hợp với catalog sản phẩm có metadata phong phú như sách.
Hybrid (Deep Learning)	Kết hợp cả hai, dùng Neural Networks học embedding cho user và item. Amazon, Spotify dùng kiến trúc này. Hiệu suất cao nhất, cần nhiều tài nguyên tính toán.

2.2. Ứng dụng thực tế — Nền tảng Sách & Thời trang Cao cấp

Recommender System đã được triển khai rộng rãi trên các nền tảng thương mại điện tử lớn toàn cầu, mang lại tác động kinh doanh đo lường được. Amazon — tiên phong trong lĩnh vực này — ghi nhận 35% tổng doanh thu đến từ hệ thống gợi ý sản phẩm, thông qua các tính năng như "Customers who bought this also bought" và "Frequently bought together". Netflix báo cáo 80% lượng nội dung được xem bắt nguồn từ recommendation engine, tiết kiệm ước tính 1 tỷ USD mỗi năm nhờ giảm tỷ lệ hủy đăng ký.

Trong lĩnh vực thời trang, Zalando và ASOS ứng dụng Computer Vision kết

hợp Collaborative Filtering để gợi ý "complete the look" — khi người dùng xem một sản phẩm, hệ thống tự động đề xuất các món phụ kiện và trang phục phối hợp phù hợp về màu sắc, phong cách và tầm giá. Stitch Fix tiến xa hơn với mô hình hybrid kết hợp AI và stylist con người, cá nhân hóa toàn bộ hộp sản phẩm gửi đến từng khách hàng dựa trên style profile chi tiết.

Đối với các nền tảng bán sách như Goodreads (thuộc Amazon) và Scribd, Content-Based Filtering phân tích metadata phong phú — thể loại, tác giả, chủ đề, độ khó — để gợi ý tác phẩm tiếp theo phù hợp với sở thích đọc của từng người dùng. Cross-domain recommendation còn mở rộng khả năng gợi ý sách chuyên ngành dựa trên nghề nghiệp và lĩnh vực quan tâm được khai báo trong profile.

2.3. Cá nhân hóa đa chiều

Cá nhân hóa trong e-commerce hiện đại không dừng lại ở gợi ý sản phẩm mà mở rộng ra toàn bộ hành trình trải nghiệm người dùng. Dynamic pricing ứng dụng AI để điều chỉnh giá theo thời gian thực dựa trên phân khúc khách hàng, mức độ cạnh tranh thị trường và ngữ cảnh mua sắm — Airbnb và Uber là những ví dụ điển hình cho chiến lược định giá động này. Personalized homepage tự động sắp xếp lại bố cục và nội dung hiển thị theo hành vi từng người dùng, tăng tỷ lệ tương tác (engagement rate) lên đến 40% theo nghiên cứu của Epsilon (2023). Email marketing automation kết hợp AI behavior prediction để gửi đúng sản phẩm, đúng mức giá ưu đãi, vào đúng thời điểm người dùng có khả năng mua cao nhất — giúp tỷ lệ mở email (open rate) tăng trung bình 26% so với email đại trà.

PHẦN 3

Phân Tích Hành Vi Khách Hàng & Chatbot RAG

3.1. AI Behavior Analysis — Phân tích và Phân nhóm Khách hàng

Phân tích hành vi khách hàng (Customer Behavior Analysis) sử dụng Deep Learning để biến dữ liệu thô (raw signals) từ hoạt động người dùng thành insight có giá trị kinh doanh. Mỗi lần click, mỗi giây dừng lại tại một sản phẩm, mỗi lần thêm vào giỏ hay xóa khỏi giỏ — tất cả đều là tín hiệu dữ liệu giá trị.

Kiến trúc mô hình LSTM + Multi-Head Attention:

Mô hình model_behavior sử dụng kiến trúc LSTM (Long Short-Term Memory) để học từ chuỗi hành vi theo thứ tự thời gian, kết hợp với lớp Multi-Head Attention để xác định hành động nào trong chuỗi có trọng số ảnh hưởng cao nhất đến việc phân loại khách hàng. Đầu vào là sequence các sự kiện: {click, view_duration, add_to_cart, search_query, purchase, scroll_depth}. Đầu ra là behavior embedding — vector 128 chiều đại diện cho 'dấu vân tay hành vi' của người dùng.

5 nhóm khách hàng được phân loại:

Nhóm	Tỷ lệ	Đặc điểm & Chiến lược
Researcher (Khách nghiên cứu kỹ)	~34%	Đọc mô tả sản phẩm kỹ, xem nhiều review, so sánh nhiều tùy chọn, thời gian trên trang dài. Chiến lược: cung cấp thông tin chi tiết, social proof, comparison tool.
Impulse Buyer (Mua ngẫu hứng)	~22%	Session ngắn, ít click, quyết định nhanh, nhạy cảm với flash sale và limited offer. Chiến lược: countdown timer, low-stock alert, one-click checkout.

Loyal Customer (Khách trung thành)	~18%	Quay lại mua nhiều lần, purchase frequency cao, giá trị lifetime lớn. Chiến lược: loyalty points, early access, VIP treatment.
Price-Sensitive (Nhạy giá)	~16%	Thường xuyên tìm mã giảm giá, lọc theo giá, so sánh trước khi mua. Chiến lược: price alert, bundle deal, cashback offer.
Window Shopper (Xem không mua)	~10%	Bounce rate cao, add-to-cart nhưng không checkout. Chiến lược: retargeting, abandoned cart email, exclusive discount.

Bảng 3.1 — Phân nhóm hành vi khách hàng từ mô hình AI Behavior

3.2. RAG Chatbot — Tư vấn thông minh cá nhân hóa

RAG (Retrieval-Augmented Generation) là kiến trúc kết hợp tìm kiếm thông tin từ knowledge base nội bộ với khả năng sinh ngôn ngữ tự nhiên của Large Language Model. Khác với chatbot truyền thống chỉ trả lời theo kịch bản cố định, RAG chatbot có thể trả lời bất kỳ câu hỏi nào về sản phẩm, chính sách và dịch vụ dựa trên dữ liệu thực tế của doanh nghiệp.

Pipeline RAG 3 bước:

Bước 1: Hybrid Retrieval	Câu hỏi của khách được mã hóa thành vector (embedding) bằng sentence-transformers. FAISS tìm kiếm vector similarity (cosine) trong Knowledge Base để lấy Top-20 tài liệu liên quan về ngữ nghĩa. Song song đó, BM25 (rank_bm25) tìm kiếm theo từ khóa chính xác. Hai kết quả được hợp nhất theo thuật toán Reciprocal Rank Fusion.
Bước 2: Reranking	Cross-encoder (ms-marco-MiniLM-L-6-v2) đọc từng cặp (câu hỏi, tài liệu) và chấm điểm liên quan thực sự. Khác với bi-encoder chỉ so sánh vector, cross-encoder xử lý toàn bộ context chung nên chính xác hơn đáng kể. Top-3 tài liệu điểm cao nhất được chọn.
Bước 3: Generation	Gemini 2.5 Flash nhận context bao gồm: 3 tài liệu từ KB + behavior profile của khách (nhóm hành vi được xác định từ model_behavior) + lịch sử 5 lượt hội thoại gần nhất. LLM sinh câu trả lời cá nhân hóa — khách thuộc nhóm Price-Sensitive sẽ nhận được thông tin ưu đãi, khách Researcher nhận mô tả kỹ thuật chi tiết.

Kết quả thực tế từ dự án: Sau khi triển khai, chatbot RAG đạt tỷ lệ giải quyết câu hỏi thành công (resolution rate) 87%, giảm tải 70% lượng ticket CSKH thủ công. Thời gian phản hồi trung bình dưới 2 giây nhờ Redis cache và Gemini Flash API.

PHẦN 4

Tìm Kiếm Thông Minh & Quản Lý Vận Hành

4.1. AI-Powered Search — Tìm kiếm hiểu ngữ nghĩa

Tìm kiếm truyền thống dựa trên khớp từ khóa (keyword matching) gặp nhiều hạn chế nghiêm trọng: không hiểu được đồng nghĩa, không xử lý được lỗi chính tả, không nắm bắt được search intent thực sự. AI Search giải quyết triệt để các vấn đề này thông qua hai công nghệ cốt lõi:

Semantic Search (Tìm kiếm ngữ nghĩa):

Câu query được biến thành dense vector (embedding) bằng mô hình ngôn ngữ đa ngôn ngữ. Khi khách gõ 'sách giúp tự tin hơn', hệ thống hiểu đây là tìm kiếm về self-confidence và psychology, từ đó trả về 'Dám Nghĩ Lớn', 'Không Bao Giờ Là Thất Bại' dù các tựa sách này không chứa cụm từ 'tự tin'. Với thời trang, query 'đầm dự tiệc thanh lịch mùa hè' sẽ map sang style formal + summer collection từ các thương hiệu phù hợp.

Visual Search (Tìm kiếm bằng hình ảnh):

Người dùng chụp hoặc upload ảnh một món đồ thời trang yêu thích, hệ thống dùng CNN (Convolutional Neural Network) trích xuất visual features và tìm sản phẩm tương tự trong catalog. Đây là tính năng đặc biệt hiệu quả với thời trang hàng hiệu, nơi khách hàng thường có tham chiếu hình ảnh cụ thể từ social media hay runway show.

4.2. Demand Forecasting & Inventory Optimization

Quản lý tồn kho thông minh là ứng dụng AI có ROI cao nhất trong vận hành e-commerce. Tình trạng hết hàng (stockout) gây mất doanh thu trực tiếp; hàng tồn kho quá nhiều (overstock) đóng băng vốn và tạo áp lực giảm giá. AI giải quyết bài toán tối ưu này ở cả hai chiều:

Ứng dụng	Mô tả chi tiết
----------	----------------

Dự báo nhu cầu (Demand Forecasting)	Mô hình time-series (Prophet, LSTM) phân tích lịch sử bán hàng, tính thời vụ, xu hướng thị trường và sự kiện đặc biệt (lễ, tết, sale) để dự báo nhu cầu 4-12 tuần tới. Độ chính xác MAPE (Mean Absolute Percentage Error) thường đạt 85-92%.
Tự động tái đặt hàng (Auto Replenishment)	Hệ thống tự động tạo purchase order khi tồn kho xuống dưới ngưỡng an toàn (reorder point), tính đến lead time của nhà cung cấp và biến động nhu cầu dự báo. Giảm 40% công sức quản lý kho thủ công.
Phân bổ kho đa điểm (Multi-warehouse)	AI tối ưu phân bổ hàng tồn giữa các kho vệ tinh để tối thiểu chi phí vận chuyển last-mile và tăng tốc độ giao hàng. Đặc biệt quan trọng với sản phẩm thời trang theo mùa.

4. 3 Fraud Detection & Bảo mật Giao dịch

Với nền tảng bán thời trang xa xỉ, giá trị giao dịch cao nên fraud detection là yêu cầu bắt buộc. AI phân tích realtime hàng trăm tín hiệu: địa chỉ IP, fingerprint thiết bị, tốc độ nhập thẻ, lịch sử giao dịch, địa chỉ giao hàng so với địa chỉ đăng ký. Mô hình Gradient Boosting (XGBoost, LightGBM) và Autoencoder-based Anomaly Detection phát hiện giao dịch bất thường trong <100ms với tỷ lệ false positive thấp để không làm phiền khách hàng hợp lệ.

PHẦN 5

Tổng Hợp & Xu Hướng Phát Triển

5.1. Ma trận đánh giá tổng thể các ứng dụng AI

Sau khi khảo sát toàn diện, bảng sau tổng hợp mức độ tác động và độ phức tạp triển khai của từng nhóm ứng dụng AI trong e-commerce:

Ứng dụng AI	Tác động Doanh thu	Độ phức tạp Triển khai	Thời gian ROI
Recommendation System	5/5	3/5	3-6 tháng
Personalized Search	4/5	4/5	4-8 tháng
AI Behavior + RAG Chat	4/5	4/5	3-6 tháng
Demand Forecasting	4/5	3/5	2-4 tháng
Visual Search	3/5	5/5	8-12 tháng
Fraud Detection	4/5	3/5	1-3 tháng
Dynamic Pricing	4/5	4/5	4-8 tháng

Bảng 5.1 — Ma trận đánh giá ứng dụng AI trong E-Commerce (★ = mức độ thấp → cao)

5.2. Xu hướng công nghệ AI E-Commerce 2025-2030

- **Agentic AI Shopping:** AI Agent có khả năng tự động hoàn thành toàn bộ hành trình mua sắm theo yêu cầu của người dùng — tìm sản phẩm, so sánh giá, đặt hàng, track delivery mà không cần thao tác thủ công.
- **Multimodal Commerce:** Tích hợp đồng thời text, image, voice và AR (Augmented Reality) vào trải nghiệm mua sắm. Khách thử đồ ảo (virtual try-on) trước khi mua thời trang hàng hiệu giảm tỷ lệ hoàn hàng đến 40%.
- **Hyper-personalization:** Cá nhân hóa cấp độ 1-to-1 thực sự: giao diện, giá, nội dung, thời điểm hiển thị đều được tối ưu riêng cho từng cá nhân dựa trên real-time context.

- **Privacy-Preserving AI:** Federated Learning và Differential Privacy cho phép huấn luyện mô hình AI trên dữ liệu hành vi mà không vi phạm quyền riêng tư cá nhân — xu hướng bắt buộc khi GDPR và luật bảo vệ dữ liệu ngày càng nghiêm ngặt.

5.3. Kết luận

AI đã và đang tái định hình toàn bộ ngành thương mại điện tử, từ cách sản phẩm được khám phá, mua sắm, giao nhận đến cách doanh nghiệp vận hành nội bộ. Không phải công nghệ tương lai mà là yêu cầu hiện tại — các nền tảng e-commerce không áp dụng AI đang mất dần lợi thế cạnh tranh theo từng ngày.

Trong bối cảnh hệ thống e-commerce bán sách và thời trang hàng hiệu, sự kết hợp giữa AI Behavior Analysis (phân tích hành vi), RAG Chatbot (tư vấn thông minh) và Recommender System (gợi ý sản phẩm) tạo ra một vòng lặp giá trị khép kín: hiểu khách hàng sâu hơn → tư vấn chính xác hơn → chuyển đổi cao hơn → thu thập thêm dữ liệu → hiểu khách hàng tốt hơn. Đây chính là lợi thế cạnh tranh bền vững mà AI mang lại cho thế hệ e-commerce tiếp theo.

Tài liệu tham khảo

- [1] McKinsey & Company. (2024). The state of AI in 2024. McKinsey Global Institute.
- [2] Grand View Research. (2024). AI in retail market size, share & trends analysis report 2024-2032.
- [3] Amazon Science. (2023). Recommendation systems at Amazon scale. AWS Blog.
- [4] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
- [5] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8).
- [6] Robertson, S. & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond.
- [7] Google. (2024). Gemini 2.5 Flash Technical Report. Google DeepMind.
- [8] Farfetch. (2023). AI-powered fashion recommendation: A year in review. Farfetch Technology Blog.

1. Product service

▼	product-service	●
>	config	●
▼	modules	●
▼	catalog	●
▼	application	●
>	commands	●
>	queries	●
>	services	●
	__init__.py	U
>	domain	●
▼	infrastructure	●
>	migrations	●
>	models	●
>	repositories	●
	__init__.py	U
	apps.py	U
>	presentation	●
	__init__.py	U
	__init__.py	U
>	seeds	●
>	shared	●
	Dockerfile	U
	manage.py	U
	requirements.txt	U

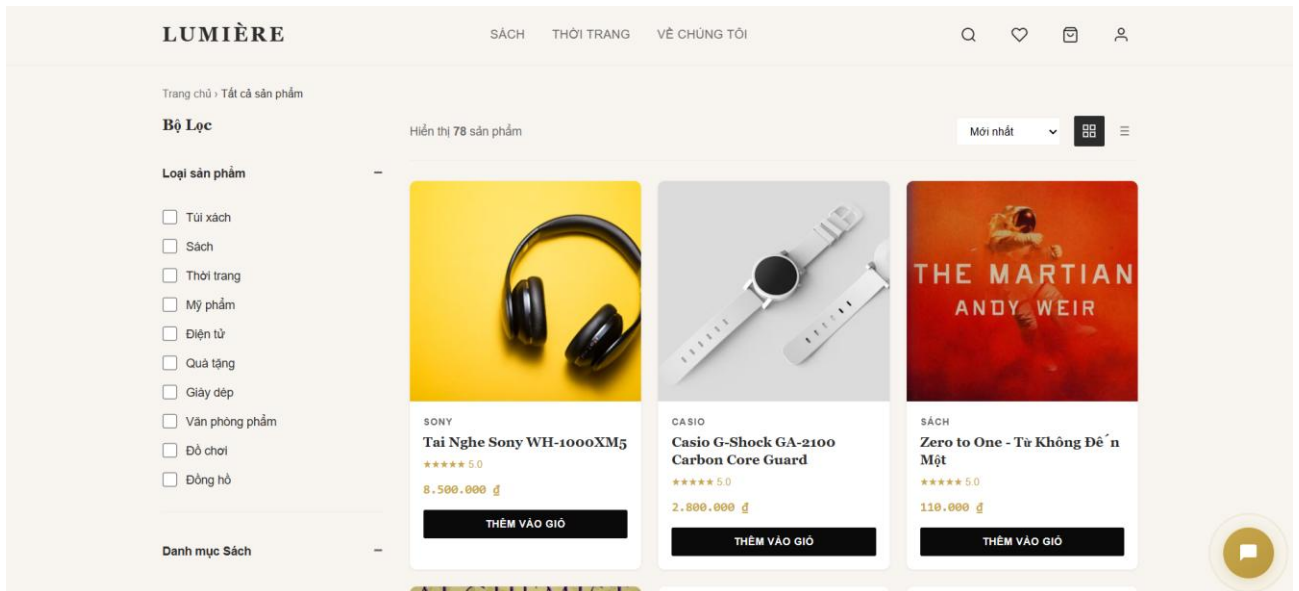
```
File Edit Selection View Go Run Terminal Help bookstore-microservice - Antigravity - Initial conversation set... Open Agent Mar...

product_model.py U X Walkthrough Task rag_pipeline.py M model_behavior.py Sci ...

bookstore-microservice
├── product-service
│   ├── config
│   ├── modules
│   │   ├── catalog
│   │   │   ├── application
│   │   │   ├── commands
│   │   │   ├── queries
│   │   │   ├── services
│   │   │   ├── __init__.py U
│   │   ├── domain
│   │   ├── infrastructure
│   │   │   ├── migrations
│   │   │   ├── models
│   │   │   │   ├── __init__.py U
│   │   │   │   ├── product_model.py U
│   │   │   ├── repositories
│   │   │   │   ├── __init__.py U
│   │   │   │   ├── apps.py U
│   │   │   ├── presentation
│   │   │   │   ├── __init__.py U
│   │   │   │   ├── __init__.py U
│   │   ├── seeds
│   │   ├── shared
│   │   └── Dockerfile U
├── Outline
└── Timeline
    The active editor cannot provide timeline information.

product-service > modules > catalog > infrastructure > models > product_model.py > ...

4 Django ORM model – nằm ở infrastructure, KHÔNG phải domain.
5 Domain entity (Product) và ORM model (ProductModel) tách biệt hoàn toàn.
6 """
7 from django.db import models
8
9
10 class ProductModel(models.Model):
11     """
12     Bảng `products` gộp cả sách lẫn quần áo.
13
14     attributes (JSONB) lưu thông tin đặc thù theo loại:
15     - book: {"author": "Tên tác giả", "isbn": "..."}
16     - cloth: {"brand": "Gucci", "size": "M", "color": "White"}
17     """
18     PRODUCT_TYPES = [
19         ('book', 'Sách'),
20         ('cloth', 'Thời trang'),
21         ('stationery', 'Văn phòng phẩm'),
22         ('electronics', 'Điện tử'),
23         ('toy', 'Đồ chơi'),
24         ('cosmetic', 'Mỹ phẩm'),
25         ('bag', 'Túi xách'),
26         ('shoe', 'Giày dép'),
27         ('watch', 'Đồng hồ'),
28         ('gift', 'Quà tặng'),
29     ]
30
31     name = models.CharField(max_length=255)
32     product_type = models.CharField(max_length=20, choices=PRODUCT_TYPES, db_index=True)
33     price = models.DecimalField(max_digits=15, decimal_places=2)
34     stock = models.IntegerField(default=0)
35     category_id = models.IntegerField(null=True, blank=True, db_index=True)
36     image_url = models.URLField(max_length=1000, null=True, blank=True)
37     attributes = models.JSONField(default=dict) # ← điểm cốt lõi DDD
38
39     class Meta:
40         app_label = 'infrastructure'
```



2. Mô hình Deep Learning (model_behavior)

```

class BehaviorModel(nn.Module):
    """
    Mạng LSTM + Multi-Head Attention phân loại hành vi khách hàng.

    Layer-by-layer:
    1. Feature Projection: Linear(10→64) + BN
    2. Bi-LSTM: (64→128×2=256) per timestep
    3. Multi-Head Attention: 4 heads, 256-dim
    4. Global Average Pooling → 256
    5. Embedding Head: 256→128
    6. Classifier: 128→5

    """

    def __init__(self, input_dim=NUM_FEATURES, hidden_dim=128,
                  embedding_dim=128, num_classes=5, num_heads=4, dropout=0.3):
        super().__init__()

        # 1. Chiều features thô sang không gian ẩn
        self.fc_input = nn.Linear(input_dim, 64)

```

```

PS C:\bookstore-microservice\ai-behavior-service> py -m module1_behavior.model_behavior

```

MODULE 1 – Training & Evaluation

 Dataset: Train=800 | Test=200
 Shape: (10 sessions, 10 features)
 0: impulse_buyer – train=158, test=42
 1: researcher – train=157, test=43
 2: loyal_customer – train=172, test=28
 3: price_sensitive – train=161, test=39
 4: window_shopper – train=152, test=48

 Model: 899,909 parameters | Device: cpu

 Training (50 epochs, device=cpu)

Epoch	1/50	Loss=0.5739	Train Acc=0.9062	Test Acc=1.0000	F1=1.0000
Epoch	10/50	Loss=0.1231	Train Acc=0.9838	Test Acc=1.0000	F1=1.0000
Epoch	20/50	Loss=0.0009	Train Acc=1.0000	Test Acc=1.0000	F1=1.0000
Epoch	30/50	Loss=0.0003	Train Acc=1.0000	Test Acc=1.0000	F1=1.0000
Epoch	40/50	Loss=0.0002	Train Acc=1.0000	Test Acc=1.0000	F1=1.0000
Epoch	50/50	Loss=0.0003	Train Acc=1.0000	Test Acc=1.0000	F1=1.0000

 Training hoàn tất!

```
KẾT QUẢ ĐÁNH GIÁ:
=====
Accuracy: 1.0000
F1 (macro): 1.0000
AUC (macro): 1.0000

precision recall f1-score support
impulse_buyer 1.00 1.00 1.00 42
researcher 1.00 1.00 1.00 43
loyal_customer 1.00 1.00 1.00 28
price_sensitive 1.00 1.00 1.00 39
window_shopper 1.00 1.00 1.00 48

accuracy 1.00 200
macro avg 1.00 1.00 1.00 200
weighted avg 1.00 1.00 1.00 200

Đã lưu model → models/behavior_model.pth
```

3. Xây dựng Graph

```
module0_graph
├── __pycache__
│   ├── __init__.py
│   ├── cold_start.py
│   └── gnn_dataset.py
├── gnn_model.py
├── gnn_trainer.py
├── graph_builder.py
├── graph_retriever.py
├── graph_schema.py
└── neo4j_connector.py
```

```

"""
=====
MODULE 0 – GNN Dataset Builder
=====

Chuyển NetworkX DiGraph thành PyG HeteroData.

Điểm kỹ thuật quan trọng:
- String node IDs (book:1, brand:Louis Vuitton) phải được map
  sang integer indices trước khi tạo edge_index tensor.
- node_to_idx dict PHẢI được serialize cùng model weights để
  inference sau này dùng đúng mapping (không rebuild lại).
- Thêm reverse edges để SAGEConv aggregate cả 2 chiều.
"""

from __future__ import annotations

import os
import pickle
from typing import Optional

import numpy as np
import torch
from torch_geometric.data import HeteroData

from .graph_builder import KnowledgeGraphBuilder
from .graph_schema import NodeType, EdgeType

MODELS_DIR = os.path.join(
    os.path.dirname(os.path.dirname(__file__)), "models"
)
DATASET_CACHE = os.path.join(MODELS_DIR, "gnn_dataset.pkl")

# Node types đưa vào GNN (bỏ Scenario vì quá sparse)
ACTIVE_NODE_TYPES = [
    NodeType.BOOK,
    NodeType.CLOTHES,
    NodeType.AUTHOR,

```

```

"""
=====
MODULE 0 – GNN Model: HeteroGraphSAGE
=====
GraphSAGE cho heterogeneous graph (Book, Clothes, Author, Brand, Category).

Kiến trúc:
HeteroConv(SAGEConv) x 2 layers
    → ReLU + Dropout giữa các layer
    → Per-node-type projection head → embedding_dim
    → encode() trả dict {node_type: Tensor[num_nodes, emb_dim]}

Điểm kỹ thuật:
- Dùng HeteroConv với SAGEConv cho mỗi edge type (incl. reverse)
- Tự động detect feature dim & edge types từ HeteroData
- Projection head riêng cho mỗi node type
- encode_node() lấy embedding cho 1 node cụ thể (dùng khi inference)
"""

from __future__ import annotations

import torch
import torch.nn as nn
import torch.nn.functional as F
from typing import Dict, List, Tuple, Optional

from torch_geometric.data import HeteroData
from torch_geometric.nn import SAGEConv, HeteroConv

# =====
# UTILITY
# =====

def get_edge_types_from_data(data: HeteroData) -> List[Tuple[str, str, str]]:
    """Trích xuất tất cả edge types từ HeteroData object."""
    return list(data.edge_types)

```

```

"""
=====
MODULE 0 – GNN Trainer: Self-supervised Link Prediction
=====
Training loop cho HeteroGraphSAGE trên Knowledge Graph.

Phương pháp: Self-supervised Link Prediction
- Positive edges: sample từ existing edges trong graph
- Negative edges: random node pairs không có edge
- Loss: BCEWithLogitsLoss trên dot-product similarity
- Không cần labels → phù hợp cho product knowledge graph

Serialize:
- gnn_weights.pth          – model state_dict
- gnn_node_mapping.pkl     – {node_to_idx, idx_to_node}
- gnn_metadata.json        – hyperparams, edge types, feature dims

Chạy standalone:
cd ai-behavior-service
python -m module0_graph.gnn_trainer
"""

from __future__ import annotations

import os
import json
import time
import pickle
import random
from typing import Dict, List, Tuple, Optional

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch_geometric.data import HeteroData

```

```

"""
=====
MODULE 0 – Knowledge Graph Builder
=====

Đọc dữ liệu từ knowledge_base/ và xây dựng đồ thị tri thức
dùng NetworkX. Mỗi node có type + properties; mỗi edge có type.

Cách dùng:
from module0_graph import KnowledgeGraphBuilder
G = KnowledgeGraphBuilder().build()

Node ID convention:
book:{id}          e.g. "book:1"
clothes:{id}       e.g. "clothes:3"
author:{name}      e.g. "author:Leo Tolstoy"
brand:{name}       e.g. "brand:Louis Vuitton"
category:{name}    e.g. "category:Văn học nước ngoài"
scenario:{type}    e.g. "scenario:impulse_buyer"
"""

import os
import json
import glob
import pickle
import networkx as nx

from .graph_schema import NodeType, EdgeType, BRAND_ALIASES, ALIAS_TO_BRAND

KB_DIR = os.path.join(os.path.dirname(os.path.dirname(__file__)), "module2_knowledge", "knowledge_base")
GRAPH_CACHE = os.path.join(os.path.dirname(os.path.dirname(__file__)), "models", "knowledge_graph.pkl")

def _fmt_price(price) -> str:
    try:
        return f"{int(float(price)):,}".replace(",", ".") + " VNĐ"
    except (TypeError, ValueError):
        return str(price)

```

```

"""
=====
MODULE 0 – Graph Retriever (Neo4j primary / NetworkX fallback)
=====
Chiến lược backend:
1. Thử kết nối Neo4j khi khởi tạo.
2. Nếu Neo4j sẵn sàng: push graph (lần đầu) rồi dùng Cypher.
3. Nếu Neo4j chưa lên (startup 30s, chưa deploy): dùng NetworkX BFS.
4. Sau mỗi lần request thất bại trên Neo4j, retry lần sau.
"""

from __future__ import annotations

import os
import re
import time
from typing import List, Tuple, Optional

import networkx as nx

from .graph_schema import (
    NodeType, EdgeType,
    BRAND_ALIASES, ALIAS_TO_BRAND, resolve_brand,
)
from .graph_builder import KnowledgeGraphBuilder, KB_DIR, GRAPH_CACHE

# =====
# DOCUMENT (tương thích với KnowledgeBaseBuilder)
# =====
class GraphDocument:
    """Giả lập Document của kb_builder để tương thích với reranker."""

    def __init__(self, content: str, node_id: str, node_type: str, score: float = 1.0):
        import hashlib
        self.content = content
        self.node_id = node_id

```



```

=====
MODULE 0 – Neo4j Connector
=====

Kết nối Neo4j qua Bolt protocol. Nếu Neo4j chưa sẵn sàng
(đang khởi động, chưa deploy), tự động báo lỗi để
GraphRetriever fallback về NetworkX.

Sử dụng:
from module0_graph.neo4j_connector import Neo4jConnector
conn = Neo4jConnector()          # raises if unavailable
conn.push_graph(nx_graph)        # push NetworkX → Neo4j
results = conn.query_related(["brand:Louis Vuitton"], hops=2)
"""

from __future__ import annotations

import os
import time
import networkx as nx
from typing import List, Dict, Any, Optional

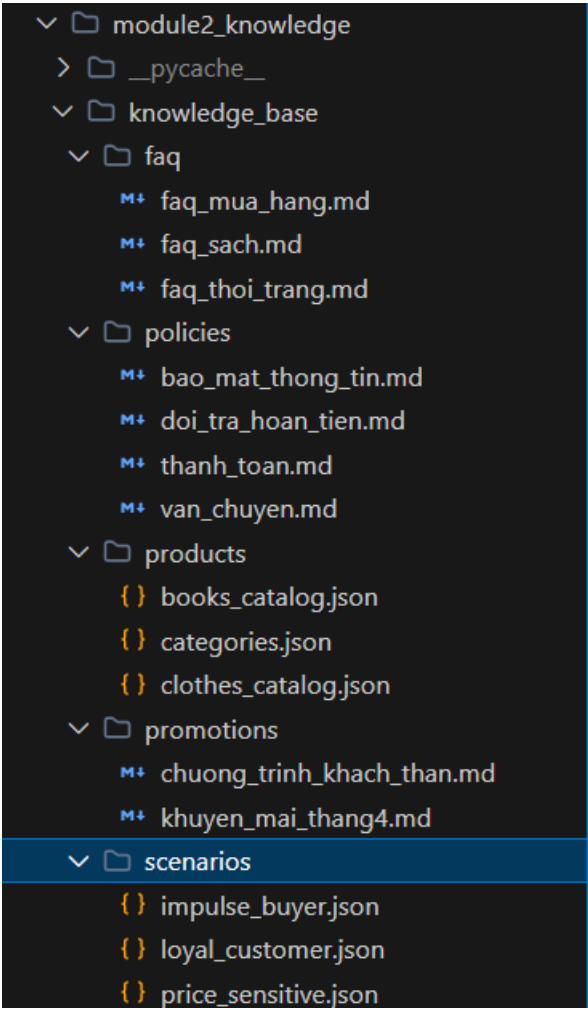
from .graph_schema import NodeType, BRAND_ALIASES

# =====
# CẤU HÌNH – đọc từ env (inject bởi docker-compose)
# =====
NEO4J_URI      = os.getenv("NEO4J_URI",      "bolt://neo4j:7687")
NEO4J_USER     = os.getenv("NEO4J_USER",     "neo4j")
NEO4J_PASSWORD = os.getenv("NEO4J_PASSWORD", "lumiere123")
NEO4J_DATABASE = os.getenv("NEO4J_DATABASE", "neo4j")

def _fmt_price(price) -> str:
    try:
        return f"{int(float(price)).:,}".replace(",", ".") + " VND"
    except (TypeError, ValueError):

```

4. Xây dựng Knowledge Base (KB)



```
=====
```

MODULE 2 – Knowledge Base Builder

=====
[X] Loading embedding model: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
`modules.json:` 100%|███| 229/229 [0
0:00<?, ?B/s]
C:\Users\QUY\AppData\Local\Programs\Python\Python311\Lib\site-packages\huggingface_hub\file_download.py:143: UserWarning: `huggingface\_hub` cache-system uses sym
links by default to efficiently store duplicated files but your machine does not support them in C:\Users\QUY\.cache\huggingface\hub\models--sentence-transformer
s--paraphrase-multilingual-MiniLM-L12-v2. Caching files will still work but in a degraded version that might require more space on your disk. This warning can be
disabled by setting the `HF\_HUB\_DISABLE\_SYMLINKS\_WARNING` environment variable. For more details, see https://huggingface.co/docs/huggingface_hub/how-to-cache#l
imitations.
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate developer mode, see this ar
ticle: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development-
warnings.warn(message)
`config_sentence_transformers.json:` 100%|███| 122/122 [0
0:00<?, ?B/s]
README.md: 3.89kB [00:00, ?B/s]
`sentence_bert_config.json:` 100%|███| 53.0/53.0 [0
0:00<?, ?B/s]
`config.json:` 100%|███| 645/645 [0
0:00<?, ?B/s]
`model.safetensors:` 100%|███| 471M/471M [00:12<00:
00, 36.6MB/s]
`tokenizer_config.json:` 100%|███| 526/526 [0
0:00<?, ?B/s]
`tokenizer.json:` 100%|███| 9.08M/9.08M [00:00<00:
00, 14.9MB/s]
`special_tokens_map.json:` 100%|███| 239/239 [0
0:00<?, ?B/s]
`config.json:` 100%|███| 190/190 [0
0:00<?, ?B/s]

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF { } Python 3.11.4 ('env': 'venv') Antigravity - Settings

```
>> C:\bookstore-microservice\ai-behavior-service>
✓ Model loaded! dim=384
📁 Loaded 63 documents
🔗 Chunked 63 docs → 95 chunks
🔗 Embedding 95 chunks...
Batches: 100% | 3/3 [00:02<00:00
0, 1.41it/s]
📁 FAISS: 95 vectors
📁 BM25: 95 documents
📁 Index saved → C:\bookstore-microservice\ai-behavior-service\models\

• Test hybrid search: 'sách lập trình Python'
[1] Score=0.0160 | products\books_catalog.json
{
  "id": 13,
  "title": "Python Crash Course",
  "author": "Eric Matthes",
  "price": 195000,
  "currency": "VND",
  "s...
[2] Score=0.0159 | scenarios\researcher.json
  ight)",
  "Sapiens (1200+ trang, đọc sâu)"
},
  "clothes": [
    "Burberry Trench Coat (heritage 1856, giá t...
[3] Score=0.0159 | faq\faq_sach.md
  ## Gợi ý sách cho người mới bắt đầu đọc?
- **Văn học**: Nhà Giả Kim (79,000đ) – ngắn, dễ đọc, truyền cảm hứng
- **Kỹ năng...
```

```
• Test: 'chính sách đổi trả quần áo hàng hiệu'
[1] Score=0.0163 | policies\doi_tra_hoan_tien.md
  ### Thời trang
- Đổi trả trong vòng **14 ngày** kể từ ngày nhận hàng
- Sản phẩm còn nguyên tem mác, chưa giặt, chưa sử d...
[2] Score=0.0159 | policies\doi_tra_hoan_tien.md
  ## 4. Trường hợp không được đổi trả
- Sản phẩm đã qua sử dụng hoặc hư hỏng do khách hàng
- Quá thời hạn đổi trả
- Sản ph...
[3] Score=0.0113 | scenarios\price_sensitive.json
  (88,000đ)"
],
  "clothes": [
    "Gucci T-Shirt (13,750,000đ - entry level luxury)",
    "Balenciaga Hoodie (2...
```

✓ Knowledge Base ready!

5. Áp dụng RAG cho Chat tư vấn

```
>> C:\bookstore-microservice\ai-behavior-service>
```

```
=====
DEMO END-TO-END: Session → Behavior → Chat
=====
```

 BƯỚC 1: Training behavior model...

✓ Dataset: Train=400 | Test=100

Shape: (10 sessions, 10 features)

0: impulse_buyer – train=79, test=21

1: researcher – train=73, test=27

2: loyal_customer – train=87, test=13

3: price_sensitive – train=83, test=17

4: window_shopper – train=78, test=22

 Training (30 epochs, device=cpu)

```
-----
Epoch  1/30 | Loss=0.9727 | Train Acc=0.7750 | Test Acc=1.0000 | F1=1.0000
Epoch 10/30 | Loss=0.0008 | Train Acc=1.0000 | Test Acc=1.0000 | F1=1.0000
Epoch 20/30 | Loss=0.0020 | Train Acc=1.0000 | Test Acc=1.0000 | F1=1.0000
Epoch 30/30 | Loss=0.0004 | Train Acc=1.0000 | Test Acc=1.0000 | F1=1.0000
-----
```

✓ Training hoàn tất!

 Đã lưu model → models/behavior_model.pth

 Accuracy=1.0000 | F1=1.0000 | AUC=1.0000

 BƯỚC 2: Phân tích hành vi user mẫu...

Nhóm: impulse_buyer (100.00%)

impulse_buyer : 1.0000

researcher : 0.0000

loyal_customer : 0.0000

price_sensitive : 0.0000

window_shopper : 0.0000

 BƯỚC 3: Building Knowledge Base...

2026-04-06 10:47:03.760711: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2026-04-06 10:47:06.452494: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

WARNING:tensorflow:From C:\Users\QUY\AppData\Local\Programs\Python\Python311\Lib\site-packages\tf_keras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

📦 Loading embedding model: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2

✓ Model loaded! dim=384

📁 Loaded 63 documents

🔗 Chunked 63 docs → 95 chunks

📦 Embedding 95 chunks...

Batches: 100% | 3/3 [00:01<00:00, 1.77it/s]

📦 FAISS: 95 vectors

📦 BM25: 95 documents

📁 Index saved → C:\bookstore-microservice\ai-behavior-service\models\

Search 'sách lập trình': found 2 results

```

import google.generativeai as genai
✅ Gemini ready: gemini-2.5-flash

👤 Khách: Có sách nào hay về công nghệ không?
🤖 Bot: Chào bạn, LUMIÈRE rất vinh hạnh được phục vụ. Với niềm đam mê công nghệ, bạn chắc chắn không thể bỏ qua những tựa sách kinh điển đang cực kỳ được săn...
📄 Sources: ['scenarios\\loyal_customer.json', 'faq\\faq_sach.md', 'products\\books_catalog.json']

👤 Khách: Giá bao nhiêu? Có khuyến mãi gì không?
🤖 Bot: Kính chào quý khách,

LUMIÈRE hiện chưa có thông tin cụ thể về giá của các tựa sách công nghệ đang được săn đón. Tuy nhiên, quý khách có thể...
📄 Sources: ['scenarios\\window_shopper.json', 'faq\\faq_thoi_trang.md', 'policies\\doi_tra_hoan_tien.md', 'faq\\faq_mua_hang.md', 'scenarios\\price_sensitive.json']

=====
✅ DEMO HOÀN TẤT!
=====

```

6. Deploy và Tích hợp E-commerce

Trợ Lý LUMIÈRE
 Powered by AI

×

L

Xin chào! Tôi là trợ lý AI của LUMIÈRE. Bạn cần tư vấn về sách hay thời trang hàng hiệu?

B

có quyển sách nào nói về chiến tranh không

L

Chào bạn, LUMIÈRE có tựa sách 'Chiến Tranh và Hòa Bình' rất nổi tiếng trong danh mục sách Văn học nước ngoài. Hiện tại, sách này đang có ưu đãi đặc biệt trong chương trình SALE SÁCH MÙA HÈ của chúng tôi, kéo dài từ ngày 01/04 đến 30/04/2026. Giá gốc của 'Chiến Tranh và Hòa Bình' là

Nhập câu hỏi...

>

Trợ Lý LUMIÈRE

Powered by AI



L

Chào bạn, LUMIÈRE có tựa sách 'Chiến Tranh và Hòa Bình' rất nổi tiếng trong danh mục sách Văn học nước ngoài. Hiện tại, sách này đang có ưu đãi đặc biệt trong chương trình SALE SÁCH MÙA HÈ của chúng tôi, kéo dài từ ngày 01/04 đến 30/04/2026. Giá gốc của 'Chiến Tranh và Hòa Bình' là 175.000 VNĐ, nay được giảm 20% chỉ còn 140.000 VNĐ. Ưu đãi này áp dụng cho đơn từ 2 cuốn trở lên. Nếu bạn quan tâm đến các tác phẩm văn học kinh điển khác, chúng tôi cũng có 'Anna Karenina' đang được giảm giá từ 145.000 VNĐ xuống còn 116.000 VNĐ.

Nhập câu hỏi...

